



NEHRU SUBHAS INSTITUTE OF TECHNOLOGY, BIHTA

Approved by AICTE, Affiliated to B.E.U. & S.B.T.E, Bihar

Board Roll No.....

Name :

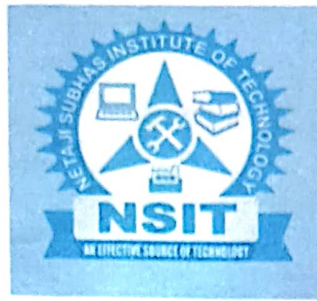
Session :

Year/Semester:

Branch :

Subject :

EXTERNAL LAB RECORD



INDEX

Lab Name: OOPS Java Lab

Faculty Incharge: Devashish Sir

Lab Incharge:

Sr. No.	Experiments Name	Date of Experiment	Completion Date	Page No.	Signature	Remarks
1	WAP to find Prime or not, Factorize, Multiplication of factors	19-12-25	19/12/25	1-2		
2	WAP in Java using Parameterized Constructors	09/01/26	09/01/26	3-4		
3	WAP in Java using Constructors or Method overriding	16/01/26	16/01/26	5-6		
4	WAP in Java Multilevel Inheritance	30/01/26	30/01/26	7-8		
5	WAP Runtime Polymorphism using method overloading	13/02/26	13/02/26	9-10		
6	WAP Exception Handling try except finally	27/02/26	27/02/26	11-12		
7						
8						
9						
10						

⇒ Write a Java program that accepts an integer from the user and performs the following operations using control structures:

* Check whether the number is Prime or not.

* Calculate the factorial of the number.

* Display the multiplication table up to 10.

Solution:-

```
import java.util.Scanner;
public class ControlProgram {
    public static void main (String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        //Prime check
        boolean isPrime = true;
        if (n <= 1) isPrime = false;
        for (int i = 2; i <= n/2; i++) {
            if (n % i == 0) {
                isPrime = false;
                break;
            }
        }
        if (isPrime)
            System.out.println("Prime Number");
    }
}
```

Output:-

Enter a number: 7

Prime Number

Factorial = 5040

Multiplication Table

$$7 \times 1 = 7$$

$$7 \times 2 = 14$$

$$7 \times 3 = 21$$

$$7 \times 4 = 28$$

$$7 \times 5 = 35$$

$$7 \times 6 = 42$$

$$7 \times 7 = 49$$

$$7 \times 8 = 56$$

$$7 \times 9 = 63$$

$$7 \times 10 = 70$$

else {

3 System.out.println("Not a Prime Number");

// Factorial

long fact = 1;

for (int i = 1; i <= n; i++)

fact *= i;

System.out.println("Factorial = " + fact);

// Multiplication Table

System.out.println("Multiplication Table: ");

for (int i = 1; i <= 10; i++)

System.out.println(n + "x" + i + " = " + (n * i));

3
3

⇒ Design a class named Employee with the following features:-

- ★ Data members: Employee ID, Name, Basic Salary.
- ★ A parameterized constructor to initialize value.
- ★ A method to calculate HRA (20%), DA (10%) and Gross Salary.
- ★ A method to display employee details.

Syntax:-

```
class Employee {  
    int id;  
    String name;  
    double basic;
```

```
    Employee (int id, String name, double basic) {  
        this.id = id;  
        this.name = name;  
        this.basic = basic;  
    }
```

```
    double calculateSalary() {  
        double hra = 0.20 * basic;  
        double da = 0.10 * basic;  
        return basic + hra + da;  
    }
```

```
    void display() {  
        System.out.println ("Employee ID: " + id);  
    }
```


Output

Employee ID: 101

Name: Amit

Basic Salary: 30000.0

Gross Salary: 39000.0

Employee ID: 102

Name: Neha

Basic Salary: 40000.0

Gross Salary: 52000.0

double calculateSalary()
double tax = 0.25 * basic;
double da = 0.10 * basic;
double gross = basic + tax + da;


```
System.out.println("Name: " + name);  
System.out.println("Basic Salary: " + basic);  
System.out.println("Gross Salary: " + calculateSalary());  
System.out.println();
```

3

```
public static void main (String [] args) {  
    Employee e1 = new Employee (101, "Anet", 30000);  
    Employee e2 = new Employee (102, "Nena", 40000);  
    e1.display();  
    e2.display();  
}
```

3

3

Create a class called calculator that demonstrates both constructors and method overloading.

- * Provide constructors to initialize zero, two and three numbers.
- * Overload a method named add() to compute the sum of
 - two integers.
 - three integers.
 - two double values.

Syntax:-

```
class Calculator {  
    calculate () {  
        System.out.println("Default constructor called");  
    }  
}
```

```
Calculator (int a, int b) {  
    System.out.println("Sum: " + (a+b));  
}
```

```
Calculator (int a, int b, int c) {  
    System.out.println("Sum: " + (a+b+c));  
}
```

```
double add (double a, double b) {  
    return a+b;  
}
```


Output.

20/10/21

Default constructor called

Sum of two integers: 15

Sum of those integers: 30

Sum of two doubles: 10.7


```
int add (int a, int b) {  
    return a+b;  
}
```

```
int add (int a, int b, int c) {  
    return a+b+c;  
}
```

3

```
public static void main (String[] args) {
```

```
    Calculator obj = new Calculator();
```

```
    System.out.println("Sum of two integers: " + obj.add(10, 10));
```

```
    System.out.println("Sum of three integers: " + obj.add(10, 10, 10));
```

```
    System.out.println("Sum of two double: " + obj.add(15.5, 15.5));
```

3

3

Write a Java program illustrating multilevel inheritance.

- ⇒ Class Person stores name and age.
- ⇒ Class Teacher inherits Person and stores Subject
- ⇒ Class Senior Teacher inherits Teacher and stores Years of experience.
- ⇒ Use the Super keyword to initialize parent class variables.
- ⇒ Display complete details.

Syntax:-

```
class Person {  
    String name;  
    int age;  
    Person (String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
class Teacher extends Person {  
    String subject;
```


Output:

Name: Raj

Age: 45

Subject: Science

Experience: 20 years.


```

Teacher (String name, int age, String subject) {
    Super (name, age);
    this.subject = subject;
}

```

3

```

class SeniorTeacher extends Teacher {
    int experience;
}

```

```

SeniorTeacher (String name, int age, String subject,
                int exp) {
    Super (name, age, subject);
    this.experience = exp;
}

```

```

void display () {
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
    System.out.println("Subject: " + subject);
    System.out.println("Experience: " + experience + "Years");
}

```

```

public static void main (String [] args) {
    SeniorTeacher st = new SeniorTeacher
        ("Raj", 45, "Mathematics", 20);
    st.display();
}

```

3

- ⇒ Create a program to demonstrate runtime polymorphism using method overriding.
- * Create a base class Shape with method area();
- * Derive classes Circle and Rectangle.
- * Override the area method in both classes.
- * Use a reference of the parent class to call overridden methods.

Syntax:-

```
class Shape {  
    void area() {  
        System.out.println("Area Calculation");  
    }  
}
```

```
class Circle extends Shape {  
    double r = 5;  
    void area() {  
        System.out.println("Area of Circle:  $\pi (3.14 \times r \times r)$ ");  
    }  
}
```


Output

Area of Circle: 78.5

Area of Rectang: 20.0

Class Rectangle extends Shape {
double l=4, b=5;

void area() {

System.out.println("Area of Rectangle: +
(l*b));

}

}

Public class Test Poly {

public static void main (String [] args) {

Shape s;

s = new Circle();

s = area();

s = new Rectangle();

s = area();

}

}

- * Write a Java program to:
 - ⇒ Accept two integers from the user.
 - ⇒ Perform division.
 - ⇒ Handle Arithmetic Exception.
- ⇒ Ensure a message "Thank you for using the program." is always printed using finally block;

Syntax:-

```
import java.util.Scanner;
```

```
class DivisionExample {  
    public static void main (String[] args) {  
        Scanner sc = new Scanner (System.in);
```

```
        try {  
            System.out.println("Enter numerator: ");  
            int a = sc.nextInt();
```

```
            System.out.println("Enter denominator: ");  
            int b = sc.nextInt();
```

```
            int result = a/b;  
            System.out.println("Result = " + result);  
        }
```


Output:

Enter numerator:-

7

Enter denominator

0

Error! Cannot divide by zero.

Thank you for using the Program.


```
catch (ArithmeticException e) {  
    System.out.println("Error: Can't divide  
by zero.");  
}
```

```
finally {  
    System.out.println("Thank you for  
using the program");  
}
```

```
}
```